Visvesvaraya Technological University

"Jnana Sangama", Belagavi-590 018

****

Seventh Semester B.E.

[As per Choice Based Credit System (CBCS) scheme]

(For Internal Circulation Only)

"Deep Learning and Reinforcement Learning (BAI701)"

(Students Manual)

Name	
USN	
Section	
Lab Batch	

****

Kalpataru Institute of Technology, Tiptur - 572 201

Department of Artificial Intelligence and Machine Learning.

AY: 2026-2027

Deep Learning and Reinforcement Learning		Semester	
Course Code	**BAI701**	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2:0	SEE Marks	50
Total Hours of Pedagogy	40 hours Theory + 8-10 Lab slots	Total Marks	100
Credits	04	Exam Hours	
Examination nature (SEE)	Theory/practical/Viva-Voce /Term-work/Others		

Course objectives:

- Understand the fundamentals of deep learning.
- Know the theory behind Convolutional Neural Networks, RNN.
- Illustrate the strength and weaknesses of many popular deep learning approaches.
- Introduce major deep learning algorithms, the problem settings, and their applications to solve real world problems

Teaching-Learning Process (General Instructions)

These are sample Strategies, which teachers can use to accelerate the attainment of the various course outcomes.

1. Black board teaching (Chalk and talk)
2. PPT and videos
3. Hands-on sessions using Python
4. Quiz/Puzzles
5. Seminars

MODULE-1

Introduction to Deep Learning

Introduction, Shallow Learning, Deep Learning, Why to use Deep Learning, How Deep Learning Works,Deep Learning Challenges,. How Learning Differs from Pure Optimization, Challenges in Neural Network Optimization.

Textbook 1: Ch 1.1 – 1.6, **Textbook 2:** 8.1,8.2

MODULE-2

Basics of Supervised Deep Learning

Introduction, Convolution Neural Network, Evolution of Convolution Neural Network, Architecture of CNN, Convolution Operation

Textbook 1: Ch 2.1 – 2.5

MODULE-3

Training Supervised Deep Learning Networks

Training Convolution Neural Networks, Gradient Descent-Based Optimization Techniques, Challenges in Training Deep Networks.

Supervised Deep Learning Architectures: LetNet-5,AlexNet

Text Book - 1 : Ch 3.2,3.4,3.5, Ch 4.2,4.3

MODULE-4

Recurrent and Recursive Neural Networks

Unfolding Computational Graphs, Recurrent Neural Network, Bidirectional RNNs, Deep Recurrent Networks, Recursive Neural Networks, The Long Short-Term Memory.Gated RNNs.

Text Book – 2: 10.1-10.3, 10.5, 10.6, 10.10

MODULE-5

Deep Reinforceme,nt Learning: Introduction, Stateless Algorithms: Multi-Armed Bandits, The Basic Framework of Reinforcement Learning, case studies.

Textbook – 3: Chapter 9: 9.1,9.2,9.3, 9.7

PRACTICAL COMPONENT OF IPCC*(May cover all / major modules)*

Sl.NO	Experiments
1	Design and implement a neural based network for generating word embedding for words in a document corpus
2	Write a program to demonstrate the working of a deep neural network for classification task.
3	Desing and implement a Convolutional Neural Network(CNN) for classification of image dataset
4	Build and demonstrate an autoencoder network using neural layers for data compression on image dataset.
5	Desing and implement a deep learning network for classification of textual documents.
6	Design and implement a deep learning network for forecasting time series data.
7	Write a program to enable pre-train models to classify a given image dataset
8	Simple Grid World Problem: Design a custom 2D grid world where the agent navigates from a start position to a goal, avoiding obstacles. Environment: Custom grid (easily implemented in Python)

Course outcomes (Course Skill Set):

At the end of the course, the student will be able to:

- CO1: Demonstrate the implementation ofdeep learning techniques
- CO2: Examine variousdeep learning techniquesfor solvingthe realworld problems
- CO3: Design and implementresearch-oriented scenario using deep learningtechniques in a team

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the theory component of the IPCC (maximum marks 50)

- IPCC means practical portion integrated with the theory of the course.
- CIE marks for the theory component are **25 marks** and that for the practical component is **25 marks**.
- 25 marks for the theory component are split into **15 marks** for two Internal Assessment Tests (Two Tests, each of 15 Marks with 01-hour duration, are to be conducted) and **10 marks** for other assessment methods mentioned in 22OB4.2. The first test at the end of 40-50% coverage of the syllabus and the second test after covering 85-90% of the syllabus.
- Scaled-down marks of the sum of two tests and other assessment methods will be CIE marks for the theory component of IPCC (that is for **25 marks)**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the theory component of IPCC.

CIE for the practical component of the IPCC

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.

- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.
- The laboratory test **(duration 02/03 hours)** after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks.**
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

SEE for IPCC

Theory SEE will be conducted by University as per the scheduled timetable, with common question papers for the course (**duration 03 hours**)

1. The question paper will have ten questions. Each question is set for 20 marks.
2. There will be 2 questions from each module. Each of the two questions under a module (with a maximum of 3 sub-questions), **should have a mix of topics** under that module.
3. The students have to answer 5 full questions, selecting one full question from each module.
4. Marks scoredby the student shall be proportionally scaled down to 50 Marks

The theory portion of the IPCC shall be for both CIE and SEE, whereas the practical portion will have a CIE component only. Questions mentioned in the SEE paper may include questions from the practical component.

Suggested Learning Resources:

Text Book:

1. M. Arif Wani Farooq Ahmad Bhat Saduf Afzal Asif Iqbal Khan, Advances in Deep Learning, Springer, 2020
2. Ian Goodfellow, Yoshua Bengio, Aaron Courville, "Deep Learning", MIT Press, 2016.
3. Charu C. Aggarwal, "Neural Networks and Deep Learning", Springer, 2018.

Reference book:

1. Bengio, Yoshua. "Learning deep architectures for AI." Foundations and trends in Machine Learning, 2009
2. N.D. Lewis, "Deep Learning Made Easy with R: A Gentle Introduction for Data Science", January 2016
3. Nikhil Buduma, "Fundamentals of Deep Learning: Designing Next-Generation Machine Intelligence Algorithms", O'Reilly publications

Web links and Video Lectures (e-Resources):

https://cedar.buffalo.edu/~srihari/

Activity Based Learning (Suggested Activities in Class)/ Practical Based learning

i) Activity Description: Review paper on Deep learning applications.

ii) Implementation:

a. **Synopsis submission:** Students will submit the synopsis based on their interest.
b. **Presentation / Demo:** Students will present their review paper along with report in specified format**.**

TABLE OF CONTENTS

Sl. No	CONTENTS	Page No
1	**INTRODUCTION**	**1-2**
2.	Design and implement a neural based network for generating word embedding for words in a document corpus	3-5
3.	Write a program to demonstrate the working of a deep neural network for classification task	6-8
4.	Desing and implement a Convolutional Neural Network(CNN) for classification of image dataset	9-12
5.	Build and demonstrate an autoencoder network using neural layers for data compression on image dataset	13-16
6.	Desing and implement a deep learning network for classification of textual documents.	17-19
7.	Design and implement a deep learning network for forecasting time series data.	20-22
8.	Write a program to enable pre-train models to classify a given image dataset	23-26
9	Simple Grid World Problem: Design a custom 2D grid world where the agent navigates from a start position to a goal, avoiding obstacles. Environment: Custom grid (easily implemented in Python)	27-30
10	VIVA QUESTIONS	31-36

INTRODUCTION

Deep Learning is a branch of Artificial Intelligence (AI) and Machine Learning (ML) that enables computers to learn from large amounts of data using artificial neural networks. Unlike traditional machine learning algorithms, deep learning models automatically learn features from raw data through multiple hidden layers, making them highly effective for solving complex problems.

Deep learning has achieved remarkable success in various fields such as computer vision, natural language processing, speech recognition, healthcare, autonomous vehicles, robotics, finance, and recommendation systems. Modern deep learning frameworks such as TensorFlow and Keras provide powerful tools for designing, training, and deploying neural network models efficiently.

Reinforcement Learning (RL) is another important area of Artificial Intelligence in which an intelligent agent learns to make decisions by interacting with an environment. The agent receives rewards or penalties for its actions and gradually learns the optimal strategy to maximize cumulative rewards. Reinforcement learning has applications in robotics, game playing, autonomous systems, and resource optimization.

This laboratory course provides practical exposure to fundamental deep learning and reinforcement learning techniques through hands-on experiments. Students will implement neural networks, convolutional neural networks (CNNs), autoencoders, text classification models, time series forecasting models, transfer learning techniques, and reinforcement learning algorithms using Python, TensorFlow, and Keras.

The experiments are designed to help students understand the concepts, implementation, and applications of deep learning models while developing practical programming skills required for solving real-world problems.

Software Requirements

Software	Version
Python	3.10 or above
TensorFlow	2.x
Keras	Latest
NumPy	Latest
Matplotlib	Latest
Scikit-learn	Latest
Jupyter Notebook / Google Colab	Latest

Hardware Requirements

Component	Requirement
Processor	Intel Core i3/i5 or above
RAM	Minimum 4 GB (8 GB Recommended)
Storage	Minimum 5 GB Free Space
Operating System	Windows 10/11, Linux, or macOS

Course Outcomes

Upon successful completion of this laboratory course, students will be able to:

- **CO1:** Demonstrate the implementation of various deep learning techniques.
- **CO2:** Apply deep learning algorithms to solve real-world problems.
- **CO3:** Design and implement research-oriented applications using deep learning and reinforcement learning techniques.

General Laboratory Instructions

Students are advised to follow the guidelines below while performing the laboratory experiments:

1. Read the experiment carefully before entering the laboratory.
2. Ensure that all required software and libraries are installed.
3. Execute the program step by step and understand the output.
4. Verify the results obtained after execution.
5. Save the program and output for future reference.
6. Maintain a neat and complete laboratory record.
7. Follow proper laboratory discipline and instructions given by the instructor.

EXPERIMENT – 1

Title

Design and Implement a Neural Network for Generating Word Embeddings for Words in a Document Corpus

Aim

To design and implement a neural network that learns dense vector representations (word embeddings) from a text corpus using TensorFlow and Keras. The generated word embeddings capture semantic relationships among words and can be utilized in various Natural Language Processing applications.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of word embeddings.
- Learn how neural networks generate vector representations of words.
- Preprocess textual data.
- Tokenize and encode text.
- Train an embedding layer.
- Visualize and interpret learned word vectors.

Libraries Required

Library	Purpose
TensorFlow	Build and train the neural network
NumPy	Numerical operations
Keras	Tokenization and Embedding layer

Installation Commands

Install TensorFlow:

pip install tensorflow

Install NumPy:

pip install numpy

Or install all required libraries together:

pip install tensorflow numpy

Algorithm

Step 1: Import the required libraries.

Step 2: Create a sample text corpus.

Step 3: Tokenize the text into words.

Step 4: Convert words into integer sequences.

Step 5: Prepare input and output data.

Step 6: Create a Sequential neural network.

Step 7: Add an Embedding layer.

Step 8: Add a Dense output layer.

Step 9: Compile the model.

Step 10: Train the model.

Step 11: Display the learned word embeddings.

Program

```
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Flatten, Dense
# Sample text
corpus = [
    "deep learning is powerful",
    "machine learning is interesting",
    "deep learning improves accuracy"
]
# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)
sequences = tokenizer.texts_to_sequences(corpus)
X = []
Y = []
for seq in sequences:
    for i in range(len(seq)-1):
        X.append(seq[i])
        Y.append(seq[i+1])
```

```
X = np.array(X)
Y = np.array(Y)
vocab_size = len(tokenizer.word_index) + 1
# Create model
model = Sequential([
    Embedding(vocab_size, 8, input_length=1),
    Flatten(),
    Dense(vocab_size, activation='softmax')
])
# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# Train model
model.fit(X, Y, epochs=200, verbose=0)
# Display embeddings
embeddings = model.layers[0].get_weights()[0]
print("Word Embeddings:\n")
for word, index in tokenizer.word_index.items():
    print(word, ":", embeddings[index])
```

.

EXPERIMENT – 2

Title

Write a Program to Demonstrate the Working of a Deep Neural Network (DNN) for a Classification Task

Aim

To design and implement a Deep Neural Network (DNN) using TensorFlow and Keras for performing a classification task on a dataset.

Objectives

After completing this experiment, students will be able to:

- Understand the architecture of a Deep Neural Network (DNN).
- Learn how to preprocess data for classification.
- Build a DNN model using TensorFlow and Keras.
- Train the model using a dataset.
- Evaluate the performance of the trained model.
- Predict the class of new input samples.

Libraries Required

Library	**Purpose**
TensorFlow	Build and train the Deep Neural Network
NumPy	Numerical computations
Scikit-learn	Load dataset and split training/testing data

Installation Commands

Install TensorFlow

pip install tensorflow

Install NumPy

pip install numpy

Install Scikit-learn

pip install scikit-learn

Install all required libraries together

pip install tensorflow numpy scikit-learn

Algorithm

Step 1: Import the required libraries.

Step 2: Load the Iris dataset.

Step 3: Split the dataset into training and testing sets.

Step 4: Build a Sequential Deep Neural Network.

Step 5: Add input, hidden, and output layers.

Step 6: Compile the model using the Adam optimizer.

Step 7: Train the model using the training data.

Step 8: Evaluate the model using the testing data.

Step 9: Predict the class labels for test samples.

Step 10: Display the accuracy of the model.

Program

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create Deep Neural Network
model = Sequential()
model.add(Dense(16, activation='relu', input_shape=(4,)))
model.add(Dense(12, activation='relu'))
```

```
model.add(Dense(3, activation='softmax'))
# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Train model
model.fit(X_train, y_train, epochs=100, verbose=0)
# Evaluate model
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("Test Accuracy:", accuracy)
# Predict first five samples
predictions = model.predict(X_test[:5])
print("\nPredicted Classes:")
for i in range(5):
    print("Sample", i+1, ":", np.argmax(predictions[i]))
```

Result

Thus, a Deep Neural Network (DNN) was successfully implemented for a classification task using the Iris dataset. The trained model accurately classified the input samples and demonstrated the effectiveness of deep learning for classification problems.

EXPERIMENT – 3

Title

Design and Implement a Convolutional Neural Network (CNN) for Classification of an Image Dataset

Aim

To design and implement a Convolutional Neural Network (CNN) using TensorFlow and Keras for image classification using the MNIST handwritten digit dataset.

Objectives

After completing this experiment, students will be able to:

- Understand the working of a Convolutional Neural Network (CNN).
- Learn how to preprocess image datasets.
- Build a CNN model using TensorFlow and Keras.
- Train the CNN model for image classification.
- Evaluate the performance of the trained model.
- Predict the class of input images.

Libraries Required

Library	Purpose
TensorFlow	Build and train the CNN model
NumPy	Numerical operations
Matplotlib	Display sample images

Installation Commands

Install TensorFlow

pip install tensorflow

Install NumPy

pip install numpy

Install Matplotlib

pip install matplotlib

Install all required libraries together

pip install tensorflow numpy matplotlib

Algorithm

Step 1: Import the required libraries.

Step 2: Load the MNIST handwritten digit dataset.

Step 3: Normalize the image pixel values.

Step 4: Reshape the images into the required input format.

Step 5: Build a Convolutional Neural Network.

Step 6: Add Convolution, Pooling, Flatten, and Dense layers.

Step 7: Compile the CNN model.

Step 8: Train the model using the training dataset.

Step 9: Evaluate the model using the testing dataset.

Step 10: Predict the class labels of test images.

Program

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.datasets import mnist

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize images
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape images
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

```
# Create CNN model
model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu',
                 input_shape=(28,28,1)))

model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(X_train, y_train,
          epochs=5,
          batch_size=32,
          verbose=1)

# Evaluate model
loss, accuracy = model.evaluate(X_test,
                                y_test,
                                verbose=0)

print("Test Accuracy:", accuracy)
```

```
# Predict first five images
predictions = model.predict(X_test[:5])


print("\nPredicted Classes:")


for i in range(5):
    print("Image", i+1, ":", np.argmax(predictions[i]))
```

Result

Thus, a Convolutional Neural Network (CNN) was successfully designed and implemented for image classification using the MNIST handwritten digit dataset. The trained model accurately classified handwritten digit images, demonstrating the effectiveness of CNNs in image recognition tasks.

EXPERIMENT – 4

Title

Build and Demonstrate an Autoencoder Network Using Neural Layers for Data Compression on an Image Dataset

Aim

To build and demonstrate an Autoencoder neural network using TensorFlow and Keras for compressing and reconstructing images from the MNIST dataset.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of Autoencoders.
- Learn how images can be compressed and reconstructed.
- Build an Autoencoder using TensorFlow and Keras.
- Train the Autoencoder using image data.
- Compare the original and reconstructed images.
- Analyze the performance of the Autoencoder for image compression.

Libraries Required

Library	**Purpose**
TensorFlow	Build and train the Autoencoder
NumPy	Numerical computations
Matplotlib	Display original and reconstructed images

Installation Commands

Install TensorFlow

pip install tensorflow

Install NumPy

pip install numpy

Install Matplotlib

pip install matplotlib

Install all required libraries together

pip install tensorflow numpy matplotlib

Algorithm

Step 1: Import the required libraries.

Step 2: Load the MNIST handwritten digit dataset.

Step 3: Normalize the image pixel values.

Step 4: Flatten the images into one-dimensional vectors.

Step 5: Build the Encoder network to compress the input image.

Step 6: Build the Decoder network to reconstruct the compressed image.

Step 7: Create the Autoencoder model.

Step 8: Compile the model using the Adam optimizer and Binary Crossentropy loss.

Step 9: Train the Autoencoder using the training images.

Step 10: Reconstruct the test images.

Step 11: Display the original and reconstructed images.

Program

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# Load MNIST dataset
(X_train, _), (X_test, _) = mnist.load_data()

# Normalize images
X_train = X_train.astype("float32") / 255
X_test = X_test.astype("float32") / 255

# Flatten images
X_train = X_train.reshape((len(X_train), 784))
X_test = X_test.reshape((len(X_test), 784))
```

```
# Encoder
input_img = Input(shape=(784,))
encoded = Dense(64, activation='relu')(input_img)

# Decoder
decoded = Dense(784, activation='sigmoid')(encoded)

# Autoencoder model
autoencoder = Model(input_img, decoded)

# Compile model
autoencoder.compile(optimizer='adam',
                    loss='binary_crossentropy')

# Train model
autoencoder.fit(X_train, X_train,
                epochs=10,
                batch_size=256,
                shuffle=True,
                verbose=1)

# Reconstruct images
decoded_imgs = autoencoder.predict(X_test)

# Display original and reconstructed images
plt.figure(figsize=(8,4))

for i in range(5):
    # Original Image
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
```

```
    plt.axis('off')
    # Reconstructed Image
    plt.subplot(2,5,i+6)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.axis('off')

plt.show()
```

Result

Thus, an Autoencoder network was successfully built and trained using the MNIST dataset. The model effectively compressed the input images into a lower-dimensional representation and reconstructed them with minimal loss of visual information.

EXPERIMENT – 5

Title

Design and Implement a Deep Learning Network for Classification of Textual Documents

Aim

To design and implement a deep learning network using TensorFlow and Keras for classifying textual documents into different categories.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of text classification.
- Learn how to preprocess textual data.
- Convert text into numerical sequences using tokenization.
- Build a deep learning model for text classification.
- Train and evaluate the model using a text dataset.
- Predict the class of new text documents.

Libraries Required

Library	Purpose
TensorFlow	Build and train the deep learning model
NumPy	Numerical computations
Keras	Tokenization, Padding, and Neural Network layers

Installation Commands

Install TensorFlow

pip install tensorflow

Install NumPy

pip install numpy

Install all required libraries together

pip install tensorflow numpy

Algorithm

Step 1: Import the required libraries.

Step 2: Load the IMDB movie review dataset.

Step 3: Restrict the vocabulary size to the most frequent words.

Step 4: Pad all review sequences to the same length.

Step 5: Build a Sequential deep learning model.

Step 6: Add an Embedding layer.

Step 7: Add a Global Average Pooling layer.

Step 8: Add Dense hidden and output layers.

Step 9: Compile the model.

Step 10: Train the model using the training dataset.

Step 11: Evaluate the model using the testing dataset.

Step 12: Display the classification accuracy.

Program

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D, Dense
# Load IMDB dataset
vocab_size = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)
# Pad sequences
max_length = 200
X_train = pad_sequences(X_train, maxlen=max_length)
X_test = pad_sequences(X_test, maxlen=max_length)
# Build the model
model = Sequential([
    Embedding(vocab_size, 16, input_length=max_length),
    GlobalAveragePooling1D(),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])
# Compile the model
```

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
# Train the model
model.fit(X_train,
          y_train,
          epochs=5,
          batch_size=512,
          validation_split=0.2,
          verbose=1)
# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("Test Accuracy:", accuracy)
```

Result

Thus, a deep learning network was successfully designed and implemented for the classification of textual documents using the IMDB movie review dataset. The trained model effectively classified reviews into positive and negative sentiment categories, demonstrating the application of deep learning techniques in text classification.

EXPERIMENT – 6

Title

Design and Implement a Deep Learning Network for Forecasting Time Series Data

Aim

To design and implement a Long Short-Term Memory (LSTM) based deep learning network using TensorFlow and Keras for forecasting time series data.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of time series forecasting.
- Learn how to preprocess sequential data.
- Build an LSTM-based deep learning model.
- Train the model using time series data.
- Predict future values of the time series.
- Evaluate the forecasting performance of the model.

Libraries Required

Library	Purpose
TensorFlow	Build and train the LSTM model
NumPy	Numerical computations
Matplotlib	Plot actual and predicted values

Installation Commands

Install TensorFlow

pip install tensorflow

Install NumPy

pip install numpy

Install Matplotlib

pip install matplotlib

Install all required libraries together

pip install tensorflow numpy matplotlib

Algorithm

Step 1: Import the required libraries.

Step 2: Create a sample time series dataset.

Step 3: Prepare input-output sequences.

Step 4: Reshape the data for the LSTM model.

Step 5: Build an LSTM network.

Step 6: Add a Dense output layer.

Step 7: Compile the model.

Step 8: Train the model using the training data.

Step 9: Predict future values.

Step 10: Display the actual and predicted values using a graph.

Program

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
# Create sample time series data
data = np.arange(1, 101)
# Prepare sequences
X = []
y = []
for i in range(len(data) - 5):
    X.append(data[i:i+5])
    y.append(data[i+5])
X = np.array(X)
y = np.array(y)
# Reshape input for LSTM
X = X.reshape((X.shape[0], X.shape[1], 1))

# Build LSTM model
```

```
model = Sequential()

model.add(LSTM(50, activation='relu', input_shape=(5,1)))
model.add(Dense(1))

# Compile model
model.compile(optimizer='adam',
              loss='mse')

# Train model
model.fit(X, y,
          epochs=200,
          verbose=0)

# Predict values
predictions = model.predict(X)

# Plot results
plt.plot(y, label='Actual Values')
plt.plot(predictions, label='Predicted Values')
plt.xlabel("Time")
plt.ylabel("Value")
plt.title("Time Series Forecasting using LSTM")
plt.legend()
plt.show()
```

Result

Thus, a Long Short-Term Memory (LSTM) based deep learning network was successfully implemented for forecasting time series data. The trained model accurately predicted future values by learning patterns from the sequential data.

EXPERIMENT – 7

Title

Write a Program to Enable Pre-trained Models to Classify a Given Image Dataset

Aim

To implement a pre-trained deep learning model using TensorFlow and Keras for classifying images from a given image dataset.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of transfer learning.
- Learn how to use pre-trained deep learning models.
- Load and preprocess image datasets.
- Build a classification model using MobileNetV2.
- Train and evaluate the model.
- Predict image classes using the trained model.

Libraries Required

Library	Purpose
TensorFlow	Build and train the model
NumPy	Numerical computations
Matplotlib	Display sample images

Installation Commands

Install TensorFlow

pip install tensorflow

Install NumPy

pip install numpy

Install Matplotlib

pip install matplotlib

Install all required libraries together

pip install tensorflow numpy matplotlib

Algorithm

Step 1: Import the required libraries.

Step 2: Load the CIFAR-10 image dataset.

Step 3: Normalize the image pixel values.

Step 4: Resize the images to 96 × 96 pixels.

Step 5: Load the pre-trained MobileNetV2 model without the top classification layer.

Step 6: Freeze the pre-trained layers.

Step 7: Add Global Average Pooling and Dense output layers.

Step 8: Compile the model.

Step 9: Train the model using the training dataset.

Step 10: Evaluate the model using the testing dataset.

Step 11: Predict the classes of test images.

Program

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense
from tensorflow.keras.models import Sequential
import numpy as np

# Load CIFAR-10 dataset
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Normalize images
X_train = X_train / 255.0
X_test = X_test / 255.0

# Resize images to 96x96
X_train = tf.image.resize(X_train, (96, 96))
```

```
X_test = tf.image.resize(X_test, (96, 96))

# Load pre-trained MobileNetV2 model
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(96, 96, 3)
)
# Freeze base model
base_model.trainable = False

# Create transfer learning model
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(10, activation='softmax')
])
# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Train model
model.fit(
    X_train,
    y_train,
    epochs=3,
    batch_size=64,
    verbose=1
)
```

```
# Evaluate model
loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)
print("Test Accuracy:", accuracy)
# Predict first five images
predictions = model.predict(X_test[:5])
print("\nPredicted Classes:")
for i in range(5):
    print("Image", i+1, ":", np.argmax(predictions[i]))
```

Result

Thus, a pre-trained **MobileNetV2** model was successfully implemented using transfer learning for image classification on the CIFAR-10 dataset. The model effectively classified images by utilizing previously learned features, demonstrating the usefulness of pre-trained deep learning models in solving image classification problems.

EXPERIMENT – 8

Title

Design a Custom 2D Grid World Environment for Reinforcement Learning

Aim

To design and implement a simple 2D Grid World environment using Q-Learning, where an agent learns to navigate from a start position to a goal while avoiding obstacles.

Objectives

After completing this experiment, students will be able to:

- Understand the basic concepts of Reinforcement Learning.
- Learn the working of Q-Learning.
- Design a simple Grid World environment.
- Train an intelligent agent using rewards and penalties.
- Find the optimal path from the start position to the goal.
- Evaluate the learning performance of the agent.

Libraries Required

Library	Purpose
NumPy	Numerical computations and Q-table operations

Installation Commands

Install NumPy

pip install numpy

Algorithm

Step 1: Import the required library.

Step 2: Create a 5 × 5 Grid World environment.

Step 3: Define the start state, goal state, and obstacle positions.

Step 4: Initialize the Q-table with zeros.

Step 5: Set the learning rate, discount factor, and exploration rate.

Step 6: Select an action using the ε-greedy policy.

Step 7: Execute the selected action and receive a reward.

Step 8: Update the Q-value using the Q-Learning equation.

Step 9: Repeat the process for multiple episodes.

Step 10: Display the optimal path learned by the agent.

Program

```
import numpy as np
# Grid size
grid_size = 5

# Start and Goal positions
start = (0, 0)
goal = (4, 4)

# Obstacles
obstacles = [(1, 2), (2, 2), (3, 1)]

# Possible actions
actions = ['Up', 'Down', 'Left', 'Right']

# Initialize Q-table
Q = np.zeros((grid_size, grid_size, len(actions)))
# Learning parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 500

# Action movements
moves = {
    0: (-1, 0),  # Up
    1: (1, 0),   # Down
    2: (0, -1),  # Left
    3: (0, 1)    # Right
}
```

```
# Training
for episode in range(episodes):
    state = start
    while state != goal:
        if np.random.rand() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(Q[state[0], state[1]])
        new_row = state[0] + moves[action][0]
        new_col = state[1] + moves[action][1]
        # Stay within grid
        if 0 <= new_row < grid_size and 0 <= new_col < grid_size:
            next_state = (new_row, new_col)
        else:
            next_state = state

        # Rewards
        if next_state in obstacles:
            reward = -10
            next_state = state
        elif next_state == goal:
            reward = 100
        else:
            reward = -1

        # Update Q-value
        Q[state[0], state[1], action] = Q[state[0], state[1], action] + \
            alpha * (reward + gamma * np.max(Q[next_state[0], next_state[1]]) -
                     Q[state[0], state[1], action])
        state = next_state
```

```
# Display optimal path
state = start
print("Optimal Path:")
while state != goal:
    print(state, end=" -> ")
    action = np.argmax(Q[state[0], state[1]])
    new_row = state[0] + moves[action][0]
    new_col = state[1] + moves[action][1]
    state = (new_row, new_col)
print(goal)
```

Result

Thus, a simple 2D Grid World environment was successfully designed and implemented using the Q-Learning algorithm. The agent learned the optimal path from the start position to the goal while avoiding obstacles through reward-based learning.

VIVA QUESTIONS

Experiment 1: Word Embeddings

1. What is a word embedding?

Answer: A word embedding is a dense vector representation of a word that captures its semantic meaning.

2. Why are word embeddings used?

Answer: They convert text into numerical vectors while preserving semantic relationships between words.

3. What is tokenization?

Answer: Tokenization is the process of splitting text into individual words or tokens.

4. What is the purpose of the Embedding layer?

Answer: It learns dense vector representations of words during model training.

5. Which library is used for tokenization?

Answer: tensorflow.keras.preprocessing.text.Tokenizer

6. What is vocabulary size?

Answer: The total number of unique words in the dataset.

7. What optimizer is used in this experiment?

Answer: Adam Optimizer.

8. What activation function is used in the output layer?

Answer: Softmax.

9. Mention two applications of word embeddings.

Answer: Text classification and sentiment analysis.

10. Name two popular word embedding techniques.

Answer: Word2Vec and GloVe.

Experiment 2: Deep Neural Network for Classification

1. What is a Deep Neural Network (DNN)?

Answer: A neural network with multiple hidden layers.

2. Why is the Iris dataset used?

Answer: It is a simple benchmark dataset for classification.

3. What are hidden layers?

Answer: Layers between the input and output layers that learn patterns.

4. What is an activation function?

Answer: A function that introduces non-linearity into the neural network.

5. Which activation function is used in hidden layers?

Answer: ReLU.

6. Which activation function is used in the output layer?

Answer: Softmax.

7. What is the loss function used?

Answer: Sparse Categorical Crossentropy.

8. Why is the dataset split into training and testing sets?

Answer: To train the model and evaluate its performance.

9. What does accuracy indicate?

Answer: The percentage of correctly classified samples.

10. What optimizer is used?

Answer: Adam.

Experiment 3: CNN for Image Classification

1. What is CNN?

Answer: A Convolutional Neural Network used mainly for image processing.

2. Why is CNN preferred for image classification?

Answer: It automatically extracts image features.

3. What is a convolution layer?

Answer: A layer that extracts features using filters.

4. What is pooling?

Answer: It reduces the size of feature maps.

5. Which pooling layer is used?

Answer: MaxPooling2D.

6. Why is the Flatten layer used?

Answer: To convert feature maps into a one-dimensional vector.

7. Which dataset is used?

Answer: MNIST.

8. How many classes are present in MNIST?

Answer: 10 classes (digits 0–9).

9. Why are pixel values normalized?

Answer: To improve training performance.

10. What activation function is used in convolution layers?

Answer: ReLU.

Experiment 4: Autoencoder

1. What is an Autoencoder?

Answer: A neural network used for data compression and reconstruction.

2. What are the two main parts of an Autoencoder?

Answer: Encoder and Decoder.

3. What is the Encoder?

Answer: It compresses the input data.

4. What is the Decoder?

Answer: It reconstructs the compressed data.

5. Which dataset is used?

Answer: MNIST.

6. Why is Binary Crossentropy used?

Answer: It is suitable for normalized image reconstruction.

7. What is latent space?

Answer: The compressed representation learned by the encoder.

8. What is image reconstruction?

Answer: Recreating the original image from compressed data.

9. Mention one application of Autoencoders.

Answer: Image compression.

10. Name another application.

Answer: Noise removal (Image Denoising).

Experiment 5: Text Classification

1. What is text classification?

Answer: Categorizing text into predefined classes.

2. Which dataset is used?

Answer: IMDB Movie Review Dataset.

3. What type of classification is performed?

Answer: Binary classification.

4. What are the two classes?

Answer: Positive and Negative.

5. Why is padding used?

Answer: To make all input sequences the same length.

6. What is the purpose of the Embedding layer?

Answer: To convert words into dense vectors.

7. Which activation function is used in the output layer?

Answer: Sigmoid.

8. Why is Binary Crossentropy used?

Answer: It is suitable for binary classification.

9. What optimizer is used?

Answer: Adam.

10. Mention one application of text classification.

Answer: Spam email detection.

Experiment 6: Time Series Forecasting

1. What is time series data?

Answer: Data collected over time intervals.

2. What is forecasting?

Answer: Predicting future values from historical data.

3. Which network is used?

Answer: LSTM.

4. Expand LSTM.

Answer: Long Short-Term Memory.

5. Why is LSTM preferred?

Answer: It remembers long-term dependencies.

6. Which loss function is used?

Answer: Mean Squared Error (MSE).

7. Name one application of time series forecasting.

Answer: Stock price prediction.

8. Mention another application.

Answer: Weather forecasting.

9. Which optimizer is used?

Answer: Adam.

10. Why are graphs plotted?

Answer: To compare actual and predicted values.

Experiment 7: Transfer Learning

1. What is transfer learning?

Answer: Reusing a pre-trained model for a new task.

2. Which pre-trained model is used?

Answer: MobileNetV2.

3. Which dataset is used?

Answer: CIFAR-10.

4. Why are pre-trained models used?

Answer: To reduce training time and improve accuracy.

5. What is ImageNet?

Answer: A large image dataset used to train deep learning models.

6. What does include_top=False mean?

Answer: It removes the original classification layer.

7. Why are the base model layers frozen?

Answer: To retain previously learned features.

8. What is Global Average Pooling?

Answer: A layer that converts feature maps into a feature vector.

9. How many classes are present in CIFAR-10?

Answer: 10 classes.

10. Mention one advantage of transfer learning.

Answer: Faster training with better performance.

Experiment 8: Reinforcement Learning – Grid World

1. What is Reinforcement Learning?

Answer: A machine learning technique where an agent learns by interacting with an environment.

2. What is an agent?

Answer: The learner that performs actions.

3. What is an environment?

Answer: The surroundings in which the agent operates.

4. What is a state?

Answer: The current position or situation of the agent.

5. What is an action?

Answer: A move made by the agent.

6. What is a reward?

Answer: Feedback received after performing an action.

7. What is Q-Learning?

Answer: A model-free reinforcement learning algorithm.

8. What is a Q-table?

Answer: A table storing Q-values for all state-action pairs.

9. What is the goal of the Grid World problem?

Answer: To reach the goal while avoiding obstacles.

10. Mention one application of Reinforcement Learning.

Answer: Robotics, autonomous vehicles, or game playing.